TÓM TẮT ĐỒ ÁN
UAV PHÁT HIỆN VÀ PHÂN LOẠI RÁC THẢI BẰNG YOLO TRÊN ROS 2 + GAZEBO

Môn Robot và Trí tuệ nhân tạo (PRRO317129) – Phương pháp, Thực nghiệm, Kết quả & Phân tích

Tóm lược: Hệ thống mô phỏng một drone (UAV) gắn camera nhìn xuống trong Gazebo Harmonic. Ảnh camera được truyền qua ROS 2 Jazzy tới node AI dùng YOLO11 để phát hiện và phân loại 4 loại rác: chai (bottle), túi nilon (plastic_bag), pin (battery) và rác hữu cơ (organic_waste). Toàn bộ hệ thống chạy trên máy tính, không cần drone hay camera vật lý. Mô hình triển khai E5 (YOLO11s) đạt mAP@0.5 = 0,937 trên tập kiểm tra trong Gazebo và 0,873 trên ảnh thật, độ trễ suy luận khoảng 8 ms/ảnh.

1. PHƯƠNG PHÁP

1.1. Dữ liệu

Bài toán là phát hiện đối tượng (object detection) 4 lớp từ góc nhìn thẳng xuống của UAV ở độ cao khoảng 3 m. Do không có sẵn bộ dữ liệu ảnh rác chụp từ drone đủ 4 lớp, đồ án kết hợp ba nguồn:

Bảng 1. Nguồn dữ liệu sử dụng

Nguồn	Nội dung	Dùng cho
moondream/waste_detection (HuggingFace)	Ảnh thật có bounding box, lấy lớp bottle, PlasticBag	Ảnh thật train/val/test + cutout chai, túi
khoaliamle/Garbage_Classification_YOLO	Ảnh phân loại, lấy battery, biological	Cutout pin, rác hữu cơ
ambientCG (CC0)	10 texture mặt đất: cỏ, đất, sỏi, nhựa đường, bê tông	Nền ảnh tổng hợp + mặt đất Gazebo

Bảng 2. Số ảnh và số bounding box theo từng tập

Tập	Số ảnh	bottle	plastic_bag	battery	organic
Tổng hợp – train	3.000	3.508	3.340	3.606	3.469
Tổng hợp – val	500	578	577	596	594
Ảnh thật – train	1.502	2.140	318	0	0
Ảnh thật – val	246	387	50	0	0
Ảnh thật – test	241	328	51	0	0
Gazebo 3 m – test	**300**	**235**	**258**	**256**	**258**
Gazebo 2 m –	120	56	31	51	38

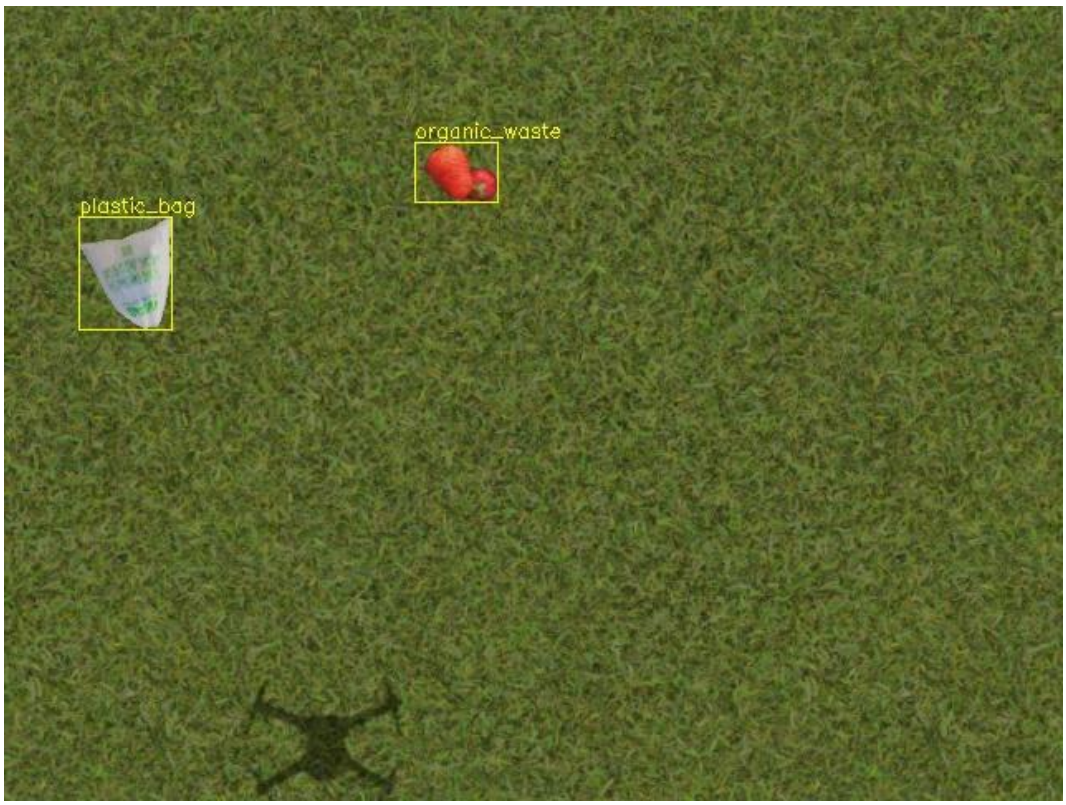
test					
Gazebo 4 m – test	120	146	126	115	161

1.2. Tiền xử lý và sinh dữ liệu

- **Tách nền:** dùng rembg (U2-Net) tách vật thể khỏi nền, lọc theo tỉ lệ phủ alpha, cắt sát vật thể.
- **Chia tập chống rò rỉ:** chia theo ảnh gốc thành train, val và sim. Tập sim chỉ dùng dựng world Gazebo, không bao giờ xuất hiện khi huấn luyện, tránh rò rỉ dữ liệu.
- **Sinh ảnh tổng hợp góc nhìn UAV:** script synth_aerial.py dán cutout lên texture mặt đất, sinh 3.000 ảnh train + 500 ảnh val 640×640; mỗi ảnh 1–9 vật thể, kích thước theo độ cao bay ~3 m, xoay ngẫu nhiên, thêm bóng đổ, nhiễu Gauss, mờ, 5% ảnh nền trống.
- **Mẫu âm khó (hard negative):** chèn bóng drone giả (không gán nhãn) vào 50% ảnh tổng hợp để mô hình học bỏ qua bóng UAV (dùng cho E6).
- **Tập kiểm tra Gazebo gán nhãn tự động:** UAV bay quét thật trong mô phỏng; nhãn sinh tự động bằng cách chiếu đường bao vật thể (ground truth trong waste_layout.json) qua mô hình camera pinhole với pose từ odometry; bỏ khung khi UAV nghiêng quá 3°.
- **Tăng cường khi huấn luyện:** mosaic (tắt 5 epoch cuối), xoay 180°, lật dọc/ngang 0,5, HSV jitter (h 0,015, s 0,5, v 0,35), scale 0,4.



Hình 1. Ảnh tổng hợp góc nhìn UAV kèm nhãn



Hình 2. Ảnh Gazebo được gán nhãn tự động bằng phép chiếu camera

1.3. Mô hình

Sử dụng họ YOLO một giai đoạn (one-stage) của Ultralytics, khởi tạo từ trọng số pretrained COCO rồi fine-tune cho 4 lớp rác. YOLO11 gồm backbone trích đặc trưng (khối C3k2, SPPF, C2PSA attention), neck kết hợp đa tỉ lệ kiểu PAN-FPN và head anchor-free tách nhánh phân loại – hồi quy hộp. Các biến thể được so sánh: YOLOv8n (đời trước), YOLO11n (nano) và YOLO11s (small, mô hình triển khai).

1.4. Hàm mất mát

Dùng hàm mất mát mặc định của Ultralytics YOLO, là tổng có trọng số của ba thành phần:

$$L = \lambda_box \cdot L_CIoU + \lambda_cls \cdot L_BCE + \lambda_dfl \cdot L_DFL \quad (\lambda_box = 7{,}5;\ \lambda_cls = 0{,}5;\ \lambda_dfl = 1{,}5)$$

- **L_CIoU (Complete IoU):** đo độ khớp hộp dự đoán với hộp thật, tính cả IoU, khoảng cách tâm và tỉ lệ cạnh.
- **L_BCE:** Binary Cross-Entropy cho xác suất từng lớp, gán mẫu dương bằng Task-Aligned Assigner.
- **L_DFL:** Distribution Focal Loss, học biên hộp dưới dạng phân phối rời rạc giúp định vị khung chính xác hơn.

1.5. Quy trình huấn luyện và suy luận

Huấn luyện: 40 epoch, ảnh 640, optimizer auto của Ultralytics (AdamW khi tổng số vòng lặp ≤ 10.000 như E1–E4, SGD khi vượt ngưỡng như E5–E6), cosine LR, early stopping patience 15, seed 0. Batch 32 cho bản n, 16 cho bản s. Mô hình tốt nhất được chọn theo tập val.

Suy luận: node ROS 2 waste_detector subscribe /camera/image_raw, chuyển ảnh bằng cv_bridge, chạy YOLO11s trên GPU với ngưỡng tin cậy 0,35, NMS IoU 0,5, ảnh 640. Kết quả được publish ra ảnh có bounding box (/waste_detection/image_annotated), mảng chuẩn vision_msgs/Detection2DArray (/waste_detection/detections) và thống kê JSON gồm số lượng từng loại, FPS, độ trễ (/waste_detection/summary).

Bảng 3. Luồng xử lý của hệ thống

Bước	Thành phần	Vai trò
1	Gazebo Harmonic + X3 UAV	Bãi cỏ 40×40 m, 64 vật thể rác, camera nhìn xuống 640×480 @15 Hz
2	ros_gz_bridge	Chuyển ảnh, odometry, lệnh vận tốc giữa Gazebo và ROS 2
3	uav_patrol	Bộ điều khiển P đơn giản, bay quét zig-zag hoặc hover
4	waste_detector	Chạy YOLO11s, publish kết quả phát hiện
5	waste_monitor, RViz2	Dashboard OpenCV tiếng Việt, hiển thị ảnh và quỹ đạo

2. THỰC NGHIỆM

2.1. Thiết lập và phần cứng

Bảng 4. Môi trường thực nghiệm

Thành phần	Cấu hình
CPU	Intel Core i5-14600KF
GPU	NVIDIA GeForce RTX 4060 8 GB
RAM	64 GB
Hệ điều hành	Windows 11 + WSL2 Ubuntu 24.04 (Gazebo render GPU qua d3d12)
Robot / mô phỏng	ROS 2 Jazzy, Gazebo Harmonic (gz-sim 8)
AI	Python 3.12, PyTorch 2.6 + CUDA 12.4, Ultralytics YOLO11, OpenCV
Thời gian huấn luyện	13–18 phút (bản n), khoảng 30 phút (bản s)

2.2. Dữ liệu đánh giá

Mỗi mô hình được đánh giá trên 5 tập: tổng hợp val (500 ảnh), ảnh thật test (241 ảnh, chỉ có chai và túi), Gazebo 3 m (300 ảnh, tập chính), Gazebo 2 m và 4 m (120 ảnh mỗi

tập, kiểm tra độ bền khi đổi độ cao bay). Các tập Gazebo dùng vật thể từ tập sim chưa từng xuất hiện khi huấn luyện.

2.3. Metrics

- mAP@0.5 và mAP@0.5:0.95 (nội suy 101 điểm như COCO) – tiêu chí chính.
- Precision, Recall, F1 và Macro-F1 tại ngưỡng tin cậy vận hành 0,35 (cùng ngưỡng với node ROS).
- Confusion matrix có lớp background để đếm false positive (FP) và false negative (FN).
- Độ trễ suy luận: median, GPU, batch 1. Không dùng accuracy vì không phù hợp bài toán phát hiện.

2.4. Baseline và thiết kế thí nghiệm

Bảng 5. Danh sách thí nghiệm

Mã	Mô hình	Dữ liệu huấn luyện	Vai trò
B0	YOLO11n COCO	Không fine-tune	Baseline mô hình có sẵn
E1	YOLOv8n	Tổng hợp	Baseline mô hình đời trước
E2	YOLO11n	Tổng hợp	So sánh kiến trúc với E1
E3	YOLO11n	Tổng hợp không bóng/nhiễu/mờ	Ablation: yếu tố chân thực
E4	YOLO11n	Tổng hợp + ảnh thật	Ablation: vai trò ảnh thật
E5	**YOLO11s**	**Tổng hợp + ảnh thật**	**Mô hình triển khai**
E6	YOLO11s	Tổng hợp + mẫu âm khó + ảnh thật	Thử nghiệm cải tiến

2.5. Quy trình đánh giá

Tất cả mô hình dùng chung một bộ đánh giá tự viết (ai/evaluate.py) với cùng tập dữ liệu, cùng ngưỡng và cùng phần cứng để so sánh công bằng. Quy trình: (1) huấn luyện bằng train.py; (2) chạy evaluate.py trên 5 tập, xuất mAP, P/R/F1 theo lớp, confusion matrix, độ trễ và lưu ảnh lỗi; (3) vẽ biểu đồ bằng plot_results.py; (4) tích hợp mô hình tốt nhất vào ROS 2 và chạy demo end-to-end trong Gazebo. Toàn bộ pipeline chạy lại được bằng scripts/run_ai_pipeline.sh.

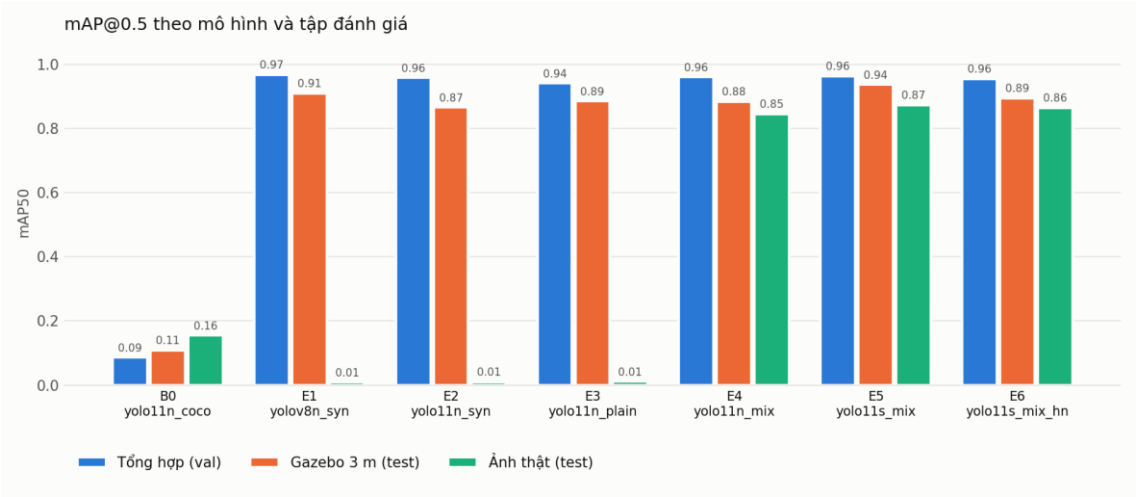
3. KẾT QUẢ VÀ PHÂN TÍCH

3.1. So sánh kết quả

Bảng 6. Kết quả tổng hợp các mô hình

Mô hình	Syn val mAP50	Gazebo 3 m mAP50	Gazebo mAP50:95	Gazebo Macro-F1	Ảnh thật mAP50	Trễ (ms)
B0 COCO	0,086	0,109	0,094	0,028	0,156	7,4

E1 YOLOv8n syn	0,967	0,909	0,547	0,863	0,009	6,3
E2 YOLO11n syn	0,958	0,865	0,505	0,836	0,010	7,8
E3 YOLO11n plain	0,941	0,885	0,484	0,856	0,011	8,3
E4 YOLO11n mix	0,960	0,884	0,531	0,825	0,846	7,9
E5 YOLO11s mix	**0,963**	**0,937**	**0,621**	**0,878**	**0,873**	**8,1**
E6 YOLO11s mix+HN	0,955	0,894	0,485	0,880	0,864	7,7



Hình 3. So sánh mAP@0.5 các mô hình trên các tập đánh giá

Bảng 7. Kết quả theo lớp của E5 trên Gazebo 3 m

Lớp	AP@0.5	Precision	Recall	F1	Số nhãn
bottle	0,974	0,849	0,932	0,888	235
plastic_bag	0,922	0,873	0,907	0,890	258
battery	0,971	0,855	0,969	0,908	256
organic_waste	0,881	0,777	0,880	0,825	258

Bảng 8. Độ bền theo độ cao bay (mAP@0.5)

Mô hình	2 m	3 m	4 m
E2 YOLO11n syn	0,912	0,865	0,858
E5 YOLO11s mix	**0,970**	**0,937**	**0,918**
E6 YOLO11s mix+HN	0,953	0,894	0,908

Mô hình chỉ được huấn luyện với kích thước vật thể tương ứng ~3 m nhưng vẫn giữ mAP@0.5 trên 0,91 ở 4 m, cho thấy khả năng tổng quát hóa tốt theo độ cao.

3.2. Thí nghiệm loại bỏ thành phần (ablation)

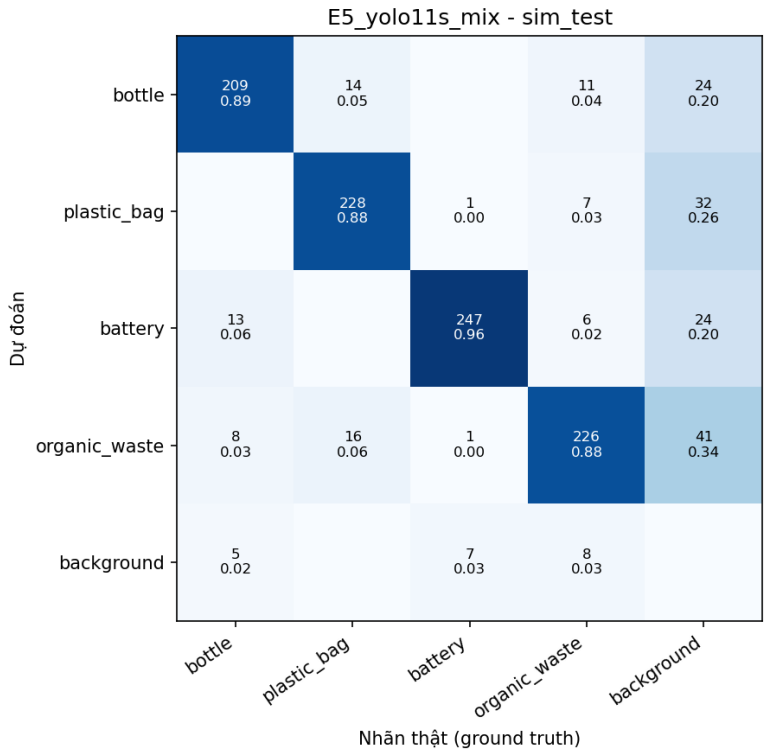
- **Vai trò fine-tune (B0 → E2):** B0 chỉ nhận được chai (AP 0,31 trên ảnh thật) và không có khái niệm túi nilon, pin, rác hữu cơ. Sau fine-tune, mAP50 trên Gazebo tăng từ 0,11 lên 0,86–0,94.

- **Bỏ ảnh thật (E4 → E2):** chỉ học ảnh tổng hợp thì mAP50 trên ảnh thật gần bằng 0 (0,010) vì ảnh thật chụp cận cảnh, khác hẳn góc nhìn UAV. Thêm 1.502 ảnh thật nâng lên 0,846 mà không làm giảm kết quả trên Gazebo (0,865 → 0,884).
- **Bỏ bóng đổ/nhiễu/mờ (E2 → E3):** mAP50:95 trên tập tổng hợp giảm từ 0,651 xuống 0,518 và mAP50 ở 4 m giảm từ 0,858 xuống 0,822. Ở 3 m E3 nhỉnh hơn (0,885 so với 0,865) nhưng với một seed khác biệt này nằm trong dao động huấn luyện. Lợi ích rõ nhất là độ khít khung và khả năng chịu thay đổi độ cao.
- **Dung lượng mô hình (E4 → E5):** YOLO11s cải thiện mọi tập (Gazebo +0,053 mAP50, +0,09 mAP50:95) với độ trễ gần như không đổi (~8 ms), nên được chọn triển khai.
- **Kiến trúc YOLOv8n vs YOLO11n (E1 vs E2):** trên tập tổng hợp tương đương; YOLOv8n nhanh hơn một chút và nhỉnh hơn trên Gazebo ở lần chạy này.

Bảng 9. Thử nghiệm mẫu âm khó trên Gazebo 3 m (E5 → E6)

Chỉ số	E5	E6
Tổng false positive (cột background)	121	75 (−38%)
FP gán nhãn organic_waste	41	19
Vật thể bị bỏ sót (FN)	20	23
Macro-F1 @0,35	0,878	0,880
mAP50 / mAP50:95	0,937 / 0,621	0,894 / 0,485

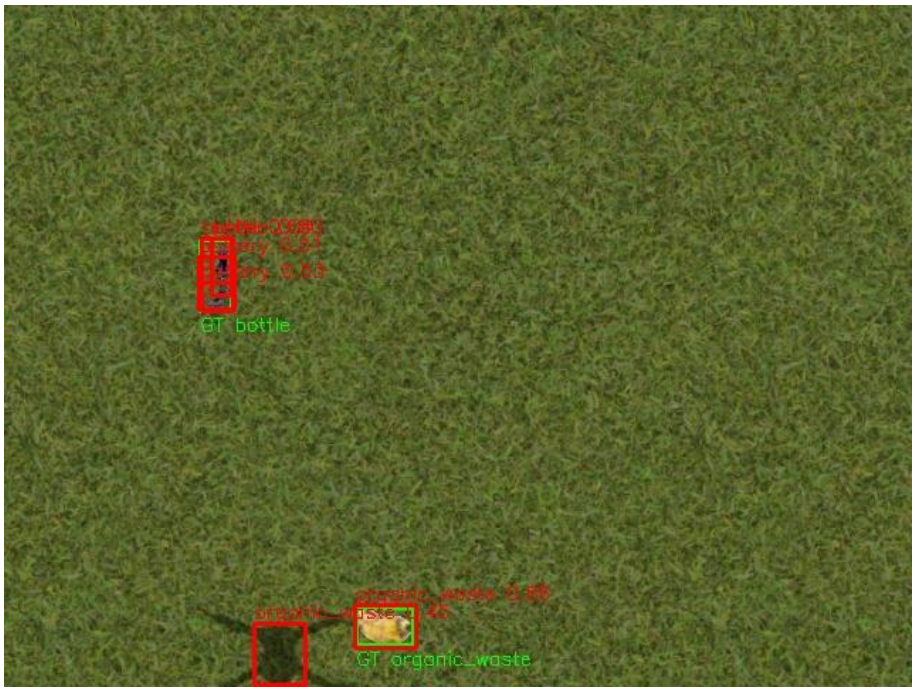
Giả thuyết bóng UAV gây FP được xác nhận một phần: FP giảm 38% nhưng mAP và độ khít khung giảm, pin bị bỏ sót nhiều hơn. Vì mAP là tiêu chí chính nên giữ E5 làm mô hình triển khai; E6 phù hợp khi ưu tiên ít báo động sai.



Hình 4. Confusion matrix của E5 trên tập Gazebo 3 m

3.3. Lỗi điển hình

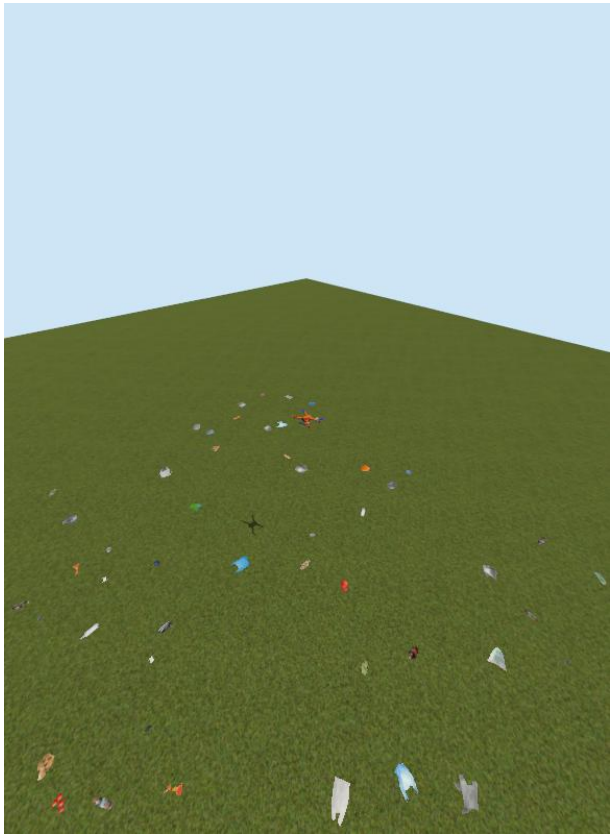
- **Nhầm lẫn màu sắc:** rác hữu cơ có màu gần với cỏ hoặc bị lẫn với túi nilon, nên organic_waste có AP thấp nhất (0,881).
- **Bóng UAV:** bóng của chính UAV in trên mặt đất bị nhận nhầm là rác hữu cơ (41 FP ở E5).
- **Vật thể nhỏ:** pin rất nhỏ (dưới 20 px) ở độ cao 4 m dễ bị bỏ sót (AP pin giảm còn 0,848).
- **Vật thể ở mép ảnh:** vật thể bị cắt ở mép ảnh khi UAV bay qua nên thiếu đặc trưng.



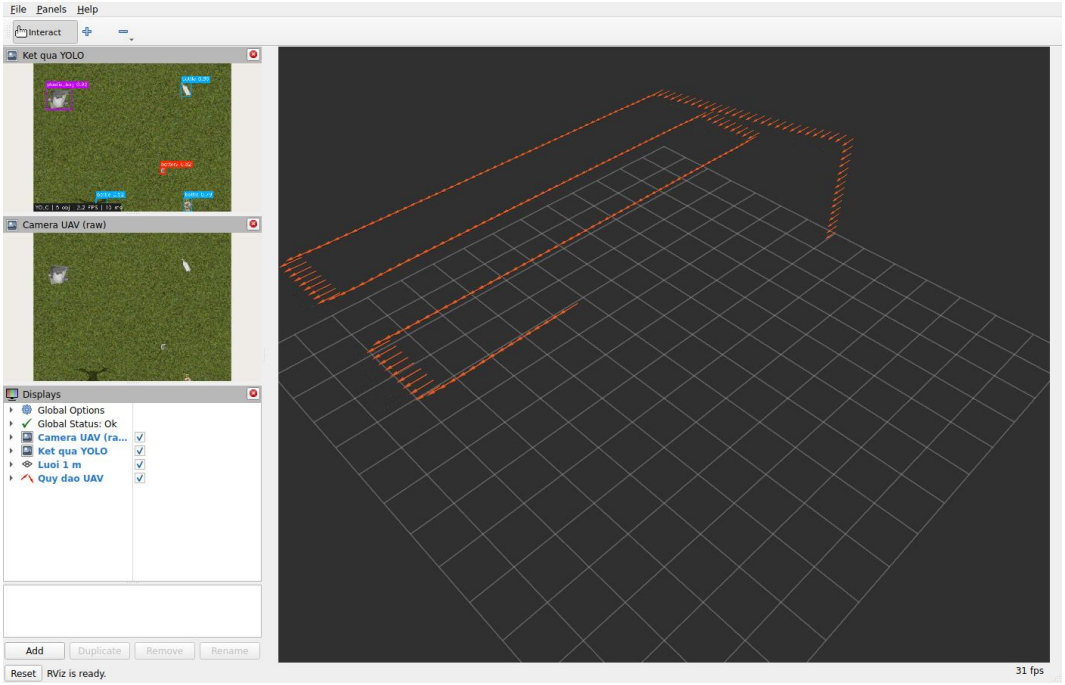
Hình 5. Ví dụ ảnh lỗi của E5 trên tập Gazebo

3.4. Tích hợp AI với robot trong mô phỏng và kết quả demo

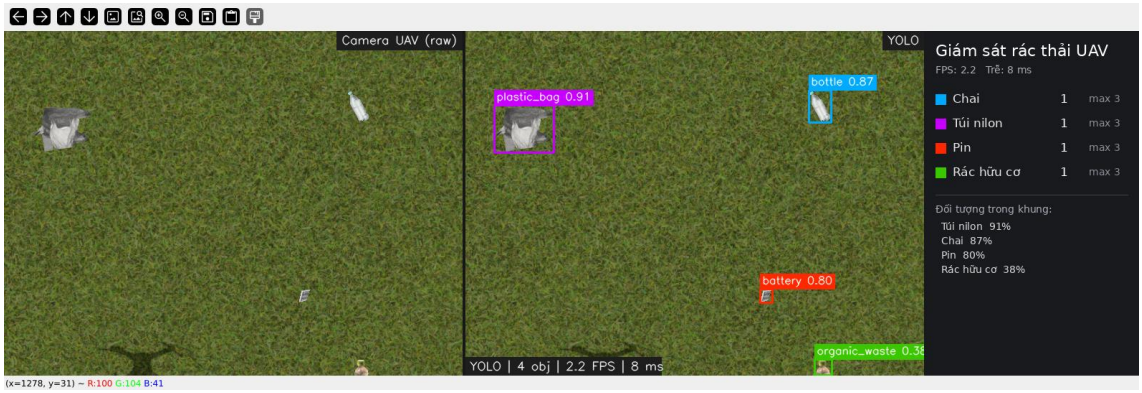
Mô hình E5 được đóng gói vào package ROS 2 uav_waste_sim. Một lệnh bash scripts/run_demo.sh khởi chạy Gazebo, bridge, UAV tự cất cánh lên 3 m và quét zig-zag trên bãi rác, node waste_detector phát hiện rác theo thời gian thực, dashboard waste_monitor hiển thị ảnh gốc, ảnh kết quả YOLO và thống kê số lượng từng loại. Video demo 85 giây được ghi từ topic /waste_detection/dashboard.



Hình 6. UAV X3 bay trên bãi rác trong Gazebo



Hình 7. RViz2 hiển thị ảnh camera, kết quả YOLO và quỹ đạo quét



Hình 8. Dashboard waste_monitor: ảnh gốc, kết quả YOLO, thống kê theo loại

Bảng 10. Hiệu năng khi chạy đủ hệ thống trên WSL2 + RTX 4060

Chỉ số	Giá trị
Độ trễ suy luận YOLO11s trong node (GPU)	8–25 ms/ảnh
Tần số ảnh camera khi bật đủ GUI	~1,5–7 Hz (giới hạn bởi render Gazebo qua WSLg)
Tần số camera khi chạy headless	~6,5 Hz (đặt 15 Hz trong SDF)
Real-time factor của Gazebo	~1,0

Checklist demo đạt đủ 7/7 tiêu chí: mở Gazebo hiển thị UAV 3D; camera tạo ảnh mô phỏng; ROS 2 nhận ảnh qua /camera/image_raw; node AI lấy ảnh từ ROS 2; phát hiện và phân loại 4 loại rác; hiển thị bounding box, nhãn, độ tin cậy; chạy demo end-to-end.

3.5. Hạn chế

- Không có drone hay camera vật lý, không tích hợp PX4/ArduPilot, không làm SLAM hay lập kế hoạch đường bay; uav_patrol chỉ là bộ điều khiển P đơn giản.
- Vật thể rác trong Gazebo là mặt phẳng dán ảnh (billboard), chưa có hình học 3D.
- Ảnh thật chỉ có nhãn cho chai và túi nilon; pin và rác hữu cơ chỉ được đánh giá trên ảnh tổng hợp và mô phỏng.
- Mỗi thí nghiệm chỉ chạy một seed nên các chênh lệch nhỏ (dưới ~0,02 mAP) chưa đủ ý nghĩa thống kê.
- Tần số camera bị giới hạn bởi render Gazebo trên WSL2, chưa đạt 15 Hz như cấu hình.